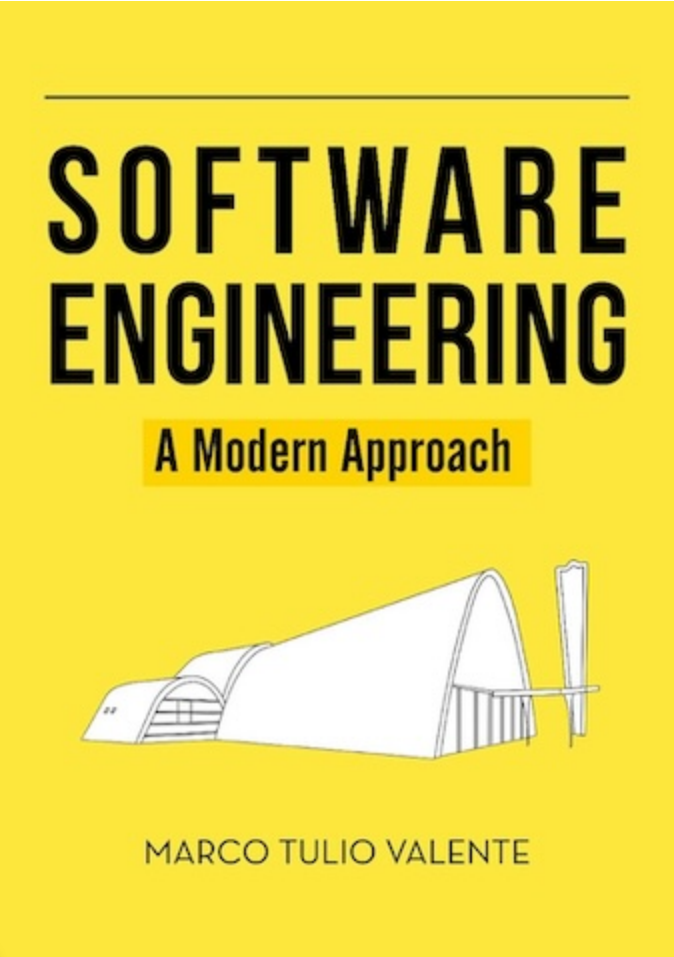


Web Version | Dark Mode | Buy on Leanpub

Software Engineering: A Modern Approach

Marco Tulio Valente

3 Requirements 🔗

> *The hardest single part of building a software system is deciding precisely what to build.* – Frederick Brooks

This chapter begins with a presentation on the importance of software requirements and their different types (Section 3.1). Next, we characterize and present the activities that comprise what we call Requirements Engineering (Section 3.2). The next four sections (Sections 3.3 to 3.6) present a variety of techniques and documents used in the specification and validation of requirements. Section 3.3 focuses on user stories, which are

the principal instruments for defining requirements in agile methods. Following that, Section 3.4 elaborates on use cases, which are more detailed documents for expressing requirements. In Section 3.5, we explore the concept of Minimum Viable Product (MVP), a popular technique for rapidly validating requirements. Finally, Section 3.6 provides insights into A/B testing, a common practice for selecting the requirements of software products.

3.1 Introduction 🔗

Requirements define what a system should do and the constraints under which it should operate. What a system should do falls under the category of **Functional Requirements**, while the constraints are described by **Non-Functional Requirements**.

To illustrate the differences between these two types of requirements more clearly, let's revisit the home-banking system example from Chapter 1. For such a system, the functional requirements include features like reporting the balance of an account, processing transfers between accounts, executing bank draft payments, and canceling debit cards, among others. In contrast, the non-functional requirements are associated to the system's quality attributes, including performance, availability, security, portability, privacy, and memory and disk usage, among others. Essentially, non-functional requirements refer to operational constraints. For example, it is not enough for our home-banking system to implement all the functions required by the bank. It also needs to have 99.9% availability, which acts as a constraint on its operation.

As Frederick Brooks emphasizes in the opening quote of this chapter, requirements specification is a critical stage in software development processes. For example, it is pointless to have a system with the best design, implemented in a modern programming language, using the best development process, and with high test coverage, if it does not meet the needs of its users. Problems in requirements specification can also incur significant costs. This is because major rework might be required when we discover—after the system is implemented and deployed—that some requirements were specified incorrectly or that important requirements were not implemented. In the worst case, there is a risk of delivering a system that will be rejected by users because it does not solve their problem.

Functional requirements are frequently specified in natural language (e.g., in English). Conversely, non-functional requirements are specified using metrics, as illustrated in the following table.

Non-Functional Req.	Metric
Performance	Transactions per second, response time, latency, throughput
Space	Storage usage, RAM, cache usage
Reliability	Uptime percentage, Mean Time Between Failures (MTBF)
Robustness	Mean Time To Recover (MTTR) after a failure, risk of data loss after a failure
Usability	User training time
Portability	Percentage of portable lines of code

Using metrics for defining non-functional requirements avoids nebulous specifications like "the system should be fast and have high availability." Instead, it is recommended to define, for example, that the system should ensure 99.99% availability and that 99% of the transactions conducted in any 5-minute window should have a maximum response time of 1 second.

Some authors, such as Ian Sommerville (link), also divide requirements into **user requirements** and **system requirements**. User requirements are high-level, non-technical, and are usually written by users in natural language. Conversely, system requirements are more detailed, precise, and defined by developers. Often, a single user requirement can expand into a set of system requirements. As an example, in our banking system, a user requirement like "the system should allow funds transfers to another bank's checking account via wire transfers" would result in system requirements that specify the protocol to be used for such transactions. Essentially, user requirements are closer to the problem, while system requirements are closer to the solution.

3.2 Requirements Engineering 🔗

Requirements Engineering encompasses activities such as the identification, analysis, specification, and validation of a system's requirements. The term engineering is used to emphasize that these activities should be performed systematically throughout the system's lifecycle, using well-defined techniques whenever possible.

The process of identifying, discovering, and understanding a system's requirements is known as **Requirements Elicitation**. Elicitation, in this context, implies drawing out the main requirements of the system from discussions and interactions with stakeholders and developers.

We can use various techniques for requirements elicitation, including conducting interviews with stakeholders, issuing questionnaires, reviewing organizational documents, organizing user workshops, developing prototypes, and analyzing usage scenarios. Other techniques rely on ethnographic studies. Ethnography, a term whose roots trace back to Anthropology, refers to studying a culture in its natural environment (*ethnos*, in Greek, means people or culture). For instance, to study a newly discovered indigenous tribe in the Amazon, an anthropologist might move to the tribe's location and spend months living among them, understanding their habits, customs, language, etc. Similarly, in the context of Requirements Engineering, ethnography is a technique for requirements elicitation that involves developers integrating into the work environment of the stakeholders and observing—typically for several days—how they perform their tasks. It's important to note that this observation is silent, meaning the developer should not interfere with or express personal views about the observed tasks and events.

Once requirements are elicited, they should be (1) documented, (2) validated, and (3) prioritized.

In Agile development, requirements are documented using **user stories**, as previously discussed in Chapter 2. However, in some projects, a **Requirements Specification Document** might be necessary. This document describes the requirements of the software to be built—including functional and non-functional requirements— typically in natural language. In the 1990s, the **IEEE 830 Standard** was proposed for writing such documents. This standard was developed within the context of Waterfall-based models, which, as we studied in Chapter 2, have a separate phase for requirements specification. The main sections of the IEEE 830 standard are presented in the following figure.

```
* External Interface Requirements
    * User Interface
    * Hardware Interface
    * Software Interface
    * Communication Interface
* Functional Requirements
    * Functional Requirement #1
    * Functional Requirement #2
    * ….
* Performance Requirements
* Design Constraints
* Other Requirements
```

Template of a requirement specification document following the IEEE 830 standard

After specification, requirements should be inspected to ensure they are correct, precise, complete, consistent, and verifiable, as described below:

- Requirements should be **correct**. For example, an incorrect computation for the savings account return in a banking system could result in either bank or client losses.
- Requirements should be **precise** to avoid ambiguity. In fact, ambiguity is more common than we might expect when using natural language. For example, consider the following condition: “to be approved, a student needs to score 60 points during the semester or score 60 points in the Special Exam and attend classes regularly.” Observe that this can be interpreted in two different ways. First: (60 points during the semester or 60 points in the Special Exam) and attend classes regularly. Alternatively, it can also be interpreted as: 60 points during the semester or (60 points in the Special Exam and regular attendance). As shown, parentheses are used to remove ambiguity in the combination of the “and” and “or” operators.
- Requirements should be **complete** to ensure that all necessary features, especially the most relevant ones, are considered and not forgotten.
- Requirements must be **consistent**. Inconsistency arises when different stakeholders have distinct expectations—for example, one stakeholder might expect an availability of 99.9%, while another believes 90% suffices.
- Requirements should be **verifiable**, meaning we can check their implementations. For example, simply stating that a system should be user-friendly is vague; how can

developers verify whether they've met the customers' expectations in this case?

Lastly, requirements must be prioritized. While the term requirements is often taken literally, i.e., as a list of mandatory features and constraints for software systems, this is not always the case. Not everything specified by the customers will be implemented in the initial releases. For instance, budget and time constraints might cause some requirements to be delayed.

Furthermore, requirements can change, as the world changes. For example, in the banking system mentioned earlier, the rules for savings account returns should be updated every time they are changed by the responsible federal agency. Thus, if a requirements-specification document exists, it should be updated, just like the source code. The ability to identify the requirements implemented by a given piece of code and vice versa (i.e., to map a particular requirement to the code implementing it) is referred to as traceability.

Before concluding, it's important to mention that Requirements Engineering is a multi-disciplinary activity. For instance, political factors might motivate certain stakeholders not to cooperate with requirements elicitation, particularly when this might threaten their status and power within the organization. Other stakeholders may simply not have time to meet with developers to explain the system's requirements. Moreover, a cognitive barrier between stakeholders and developers might also impact the elicitation of requirements. For example, stakeholders, who are typically seasoned experts, might use specialized terminology, unfamiliar to developers.

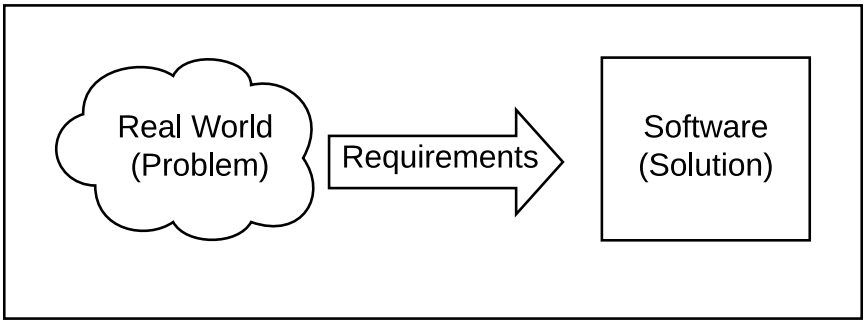
Real World: To understand the challenges faced in Requirements Engineering, in 2016, a group of about two dozen researchers conducted a survey with 228 software-developing companies spread across 10 countries (link). When asked about the main problems faced in requirements specification, the ten most common answers were as follows, including the percentage of companies that cited each problem:

- Incomplete or undocumented requirements (48%)
- Communication flaws between developers and customers (41%)
- Constantly changing requirements (33%)
- Abstractly specified requirements (33%)
- Time constraints (32%)
- Communication flaws among team members (27%)
- Difficulty in distinguishing requirements from solutions (25%)
- Insufficient support from customers (20%)

- Inconsistent requirements (19%)
- Weak access to customers' or business information (18%)

3.2.1 Topics of Study 🔗

The following figure summarizes our studies on requirements so far, showing how requirements act as a bridge that links a real-world problem with software that solves it. We will use this figure to motivate and introduce the topics we will study in the rest of this chapter.



Requirements are the bridge between real-world problems and their software solutions

First, the figure illustrates a common situation in Requirements Engineering: systems whose requirements change frequently or whose users cannot accurately specify what they want in the system. We've already mentioned this situation in Chapter 2, when we discussed Agile Methods. As you may recall, when requirements change frequently, and the system is non-mission-critical, it is not recommended to invest years in drafting a detailed requirements document. There's a risk that the requirements may become outdated before the system is finalized—or that a competitor can anticipate and build an equivalent system and dominate the market. In such cases, as we recommended in Chapter 2, we should use lightweight requirement specification documents—such as **user stories**—and incorporate a customer representative into the development team, to clarify and explain the requirements to the developers. Given the importance of such scenarios—systems with evolving, but non-critical requirements—we will start by studying user stories in Section 3.3.

On the other hand, some systems have relatively stable requirements. In these cases, it might be worth investing in a detailed requirement specification. For example, certain companies prefer to document all of the system's requirements prior to development. Additionally, requirements may be demanded by certification organizations, especially for systems that deal with human lives, such as systems in the medical, transportation, or

military domains. In Section 3.4, we will study **use cases**, which are comprehensive documents for specifying requirements.

A third scenario arises when we do not know if the proposed "problem" truly warrants a solution. In this case, we might collect all the requirements of this "problem" and implement a system that solves it. However, uncertainty remains about whether the system will gain traction and attract users. In these scenarios, an interesting approach is to take a step back and first test the relevance of the problem we intend to solve with software. One possible test involves building a **Minimum Viable Product** (**MVP**). An MVP is a functional system that can be used by real customers. However, it only includes the features necessary to prove its market feasibility, i.e., its ability to solve a problem faced by some customers. Given the contemporary importance of such scenarios—software for solving problems in unknown or uncertain markets—we will study MVPs in more detail in Section 3.5.

3.3 User Stories 🔗

Requirements documents produced by waterfall development processes can often amount to hundreds of pages that sometimes require more than a year to complete. These documents often encounter the following problems: (1) they may become obsolete as requirements change during development; (2) descriptions in natural language tend to be ambiguous and incomplete; thus, developers often need to go back and talk to the customers during the implementation phase to clarify doubts; (3) when these conversations do not happen, the risks are even higher: at the end of the implementation, customers may conclude they do not want the system anymore, as their priorities have changed, their vision of the business has changed, or the internal processes of their company have changed, among other reasons. Therefore, a long initial phase of requirements specification is increasingly rare, at least in the case of commercial systems, which are the focus of this book.

The professionals who proposed agile methods recognized—or suffered from—such problems and proposed a pragmatic technique to solve them, known as **User Stories**. As described by Ron Jeffries in a book on Agile Development (link), a user story has three parts, also termed the three Cs:

> User Story = Card + Conversations + Confirmation

Let's explore each of these parts of a story:

- **Card**, which is used by customers to write, in their own language and in a few sentences, a feature they hope to see implemented in the system.
- **Conversations** between customers and developers to allow the former to gain a better understanding of what is detailed on each card. As stated before, the agile methods take a pragmatic approach to requirements: as textual requirements specifications are subject to problems, they have been eliminated and replaced by verbal communication between developers and customers. Moreover, agile methods—as we studied in Chapter 2—recommend that a customer representative, also known as a Product Owner or Product Manager, should be part of the team.
- **Confirmation**, which is a high-level test—specified by the customer—to verify whether the story was implemented as expected. It is not an automated test, like a unit test, but rather a textual description of the scenarios, examples, and test cases that the customer will use to confirm the implementation of the story. These tests are also called **acceptance tests**. They should be written as soon as possible, preferably at the beginning of a sprint. Some authors recommend writing them on the back of the user story cards.

For this reason, requirement specifications using stories do not consist of just two or three sentences, as some critics of agile methods may claim. Instead, a user story should be interpreted as follows: The story written on the card serves as a reminder from the customer's representative to the developers. By creating this reminder, the representative indicates that they would like to see a feature implemented in an upcoming sprint. Furthermore, they commit to being available during the sprints to explain the feature to the developers. Finally, they will consider the story implemented only if it meets the confirmation tests they have specified.

From a developer's standpoint, the process works like this: The customer representative asks us to implement the story summarized on a card. Then, we implement it in a future sprint. During this process, we can rely on the support of the customer representative to discuss and clarify any doubts about the story. The representative also defines the tests they will use at the sprint review meeting to determine if the story is implemented correctly. We further agreed that the representative cannot change their mind at the end of the sprint and use an entirely different test to assess our implementation.

In essence, when employing user stories, requirements engineering becomes a continuous activity occurring every day throughout the sprints. The traditional requirements document with hundreds of pages is replaced by regular conversations between developers and the customer representative. User stories emphasize verbal engagement over written

communication, thus aligning with the following principles of the Agile Manifesto: (1) individuals and interactions over processes and tools; (2) working software over comprehensive documentation; (3) customer collaboration over contract negotiation; and (4) responding to change over following a plan.

In more specific terms, user stories should have the following properties (whose initials form the acronym INVEST):

- **Independent:** It should be possible to implement any two stories in any order. Ideally, there should be no dependencies between stories.
- **Negotiable:** As mentioned earlier, stories (the card) are invitations for conversations between customers and developers during a sprint. Both parties should be open to changing and adapting their opinions as a result of these discussions. Developers should be open to implementing details not expressed on or that do not fit on the story cards. Customers should also consider technical arguments from developers, such as those regarding the complexity of implementing some aspects of the story as initially planned.
- **Valuable:** Stories should add value to the customers' business. Stories are proposed, written, and ranked by the customers according to the value they add to their business. Consequently, the idea of a technical story—such as "the system must be implemented in JavaScript, using React and Node.js"—is inappropriate.
- **Estimable:** It should be possible to estimate the size of a story, i.e., to define the effort needed to implement it. Typically, this requires the story to be small, as we will discuss later. Estimation also becomes easier when the developers have experience in the system's domain.
- **Small:** Complex and large stories—also known as **epics**—can exist, but they should be placed at the bottom of the backlog, indicating they will not be implemented soon. On the other hand, stories at the top of the backlog should be short and small to facilitate understanding and estimation. For a one-month sprint, it should be possible to implement any story in less than one week, for example.
- **Testable:** Stories should have clear acceptance tests. For example, "the customer can pay by credit card" is testable, provided we know the kind of credit card that will be used. Conversely, a story like "a customer should not have to wait too long to have their purchase confirmed" is not testable. This is because it lacks a clear acceptance test.

It is also recommended to define the key users who will interact with the system before writing stories. This approach helps avoid stories that only serve certain users. Once you have defined these **user roles** (or **personas**), stories are commonly written in the following format:

> As a [user role], I want to [do something with the system]

We will show examples of stories in this format in the next section. But first, we would like to mention that a **story writing workshop** is usually carried out at the inception of a software project. This workshop brings together the system's main users to discuss the system's objectives, main features, and other key aspects. The workshop can last up to a week, depending on the project's size and importance. By its conclusion, we should have a list of user stories for implementation over multiple sprints.

3.3.1 Example: Library Management System

In this section, we provide examples of user stories for a library management system. These stories are associated with three user types: students, instructors, and library staff.

First, we present stories suggested by students (see below). Any library user can perform the operations described in these stories. Note that the stories are brief and do not elaborate on how each operation should be implemented. For example, one of the stories states that students should be able to search for books. However, many details are omitted, including search criteria, available filters, limits on the number of search results, and the layout of search and results pages. Nonetheless, we should remember that a story is essentially a commitment: the customer representative ensures they will be available to clarify these details with developers during the sprint in which the story is implemented. When working with user stories, this verbal interaction between developers and the customer representative is crucial to successful requirements specification and implementation.

> As a student, I want to borrow books.
>
> As a student, I want to return books I have borrowed.
>
> As a student, I want to renew my book loans.

> As a student, I want to search for books.
>
> As a student, I want to reserve books that are currently on loan.
>
> As a student, I want to receive notifications about new acquisitions.

Now, we present the stories suggested by the instructors:

> As an instructor, I want to borrow books for an extended period of time.
>
> As an instructor, I want to recommend books for acquisition.
>
> As an instructor, I want to donate books to the library.
>
> As an instructor, I want to return books to other campus libraries.

Although these stories originate from instructors, this does not mean they are exclusive to this user group. For example, during the sprint, the customer representative (or Product Owner) may consider making the donation feature available to all users.

The last story proposed by instructors—allowing books to be returned to any university library—can be classified as an **epic**, that is, a complex story. This story refers to a scenario where an instructor borrows a book from the central library but wants to return it to a departmental library, or vice versa. Implementing this story is more complex as it requires integrating different library systems and having staff members transport the books back to their original locations.

Finally, we share the stories proposed by the library staff members, typically concerning library organization and ensuring its seamless operation:

> As a staff member, I want to register new users.
>
> As a staff member, I want to add new books to the system.
>
> As a staff member, I want to remove damaged books from the system.
>
> As a staff member, I want to access statistics about the collection.
>
> As a staff member, I want the system to send reminders to users with overdue books.

> As a staff member, I want the system to apply fines for late book returns.

To confirm the implementation of the search story, the customer representative defined the following test scenarios:

> Search for books using an ISBN.
>
> Search for books using an author's name.
>
> Search for books using a title.
>
> Search for books added to the library on or after a specific date.

The correct implementation of these searches will be demonstrated during the Sprint Review meeting, assuming the team is using Scrum.

As previously mentioned, acceptance tests are specified by the customer representative (or Product Owner). This practice helps prevent a scenario known as **gold plating**. In Requirements Engineering, this term describes the situation where developers independently decide to elaborate on certain stories—or requirements, more generally—without the customer's input. Metaphorically, developers are embellishing stories with layers of gold, which does not generate value for users.

3.3.2 Frequently Asked Questions 🔗

Before we wrap up, and as usual in this book, let's answer some questions about user stories:

Can we specify non-functional requirements using stories? This is a challenging issue when using agile methods. Indeed, the customer representative (or Product Owner) may write a story stating that "the system's maximum response time is one second." However, it doesn't make sense to allocate this story to a given sprint, as it should be a concern in each sprint of the project. Therefore, the best solution is to allow (and encourage) the PO to write stories about non-functional requirements, but use them primarily to reinforce the "done criteria" for stories. For example, for the implementation of a story to be considered complete, it should pass a code review aimed at detecting performance problems. Before the code moves to production, a performance test can also be executed to ensure that the

non-functional requirements are being met. In summary, one can—and should—–write stories about non-functional requirements, but they do not go into the product backlog. Instead, they are used to refine the “done criteria” for stories.

Is it possible to create stories for studying a new technology? Conceptually, the answer is that one should not create stories exclusively for knowledge acquisition, as stories should always be written and prioritized by customers, and they should provide business value. Therefore, we should not break this principle by allowing developers to create a story solely to “study the use of framework X in the web interface implementation.” However, this study could be a task associated with the implementation of a specific story. In agile methods, tasks for knowledge acquisition or for creating a proof-of-concept implementation are called **spikes**.

3.4 Use Cases 🔗

Use Cases are textual documents used to specify requirements. As this section will explore, they offer more detailed descriptions than user stories and are typically used in Waterfall-based methods. Developers, who are also referred to as Requirements Engineers during this phase of development, write the use cases. They can rely on methods such as interviews with users for this purpose. Although use cases are written by developers, users should be able to read, understand, and validate them.

Use Cases are written from the perspective of an **actor** interacting with the system to achieve specific objectives. Typically, the actor is a human user, although it can also be another software or hardware component. In all cases, the actor is an entity external to the system.

A use case enumerates the actions that an actor should perform to complete a specific operation. Specifically, a use case defines two lists of steps. The first list represents the **main flow**, which consists of steps required to successfully complete an operation. This main flow describes a scenario in which everything goes well, also known as the “happy path”. The second list defines **extensions** to the main flow, which represent alternatives for executing particular steps of the main flow or for handling errors. Both flows should be implemented by the system in later stages of development. The following example shows a use case that specifies a transfer between accounts in a banking system.

> **Transfer Values between Accounts**

> **Actor:** Bank Customer
>
> **Main Flow:**
>
> 1 - Authenticate Customer
>
> 2 - Customer enters destination account and branch
>
> 3 - Customer enters the amount for transfer
>
> 4 - Customer specifies the transfer date
>
> 5 - System executes the transfer
>
> 6 - System asks if the customer wants to make another transfer
>
> **Extensions:**
>
> 2a - If the account or branch is incorrect, system requests new account and branch
>
> 3a - If transfer amount exceeds current balance, system requests new amount
>
> 4a - Transfer date must be the current date or within one year in the future
>
> 5a - If the transfer date is the current date, system processes transfer immediately
>
> 5b - If the transfer date is in the future, system schedules the transfer

We will now use this example to highlight other relevant points about use cases. First, every use case must have a name that starts with a verb in the base form. Second, it should identify the main actor of the use case. Additionally, a use case can include another use case. In our example, step 1 of the main flow includes the “Authenticate Customer” use case. The syntax for inclusions is simple: the included use case’s name is underlined. The semantics are straightforward: all steps of the included use case must be executed before proceeding. This behavior is similar to macro expansion in programming languages.

Finally, we will discuss the extensions, which serve two objectives:

- To detail a step in the main flow. In our example, we used extensions to specify that the transfer must be executed immediately on the current date (extension 5a). Otherwise, the system should schedule the transfer for the specified date (extension 5b).

- To handle errors, exceptions, cancellations, etc. In our example, we used an extension to specify that the system should request a new amount if there is insufficient balance for the transfer (extension 3a).

Because of the existence of extensions, we recommend avoiding decision statements such as `if` in the main flow of use cases. When a decision between two normal behaviors is necessary, it should be defined as an extension. This is one of the reasons why extensions in real-world use cases often have more steps than the main flow. Our simple example nearly illustrates this point, with five extensions compared to six main steps.

Use case descriptions may occasionally include additional sections, such as: the purpose of the use case; pre-conditions, which define what must be true before the use case is executed; post-conditions, which specify what must be true after the use case is executed; and related use cases.

To conclude, here are some good practices for writing use cases include:

- Use cases should be written in simple and accessible language. A common recommendation is to "write use cases as if you were in early elementary school." Ideally, each step should describe the main actor performing a task, using an active verb. For example, "The customer inserts the card into the ATM." Similarly, when the system performs an action, write a step like "The system validates the inserted card."

- Use cases should be concise, with few steps, especially in the main flow, to facilitate understanding. Alistair Cockburn, the author of a well-known book on use cases (link, page 91), recommends a maximum of nine steps in the main flow. He states the following: "I rarely encounter a well-written use case with more than nine steps in the main success scenario." Therefore, if you are writing a use case and it becomes complex, consider breaking it down into smaller ones. Another alternative is to group related steps. For example, the steps "User enters login" and "User enters password" can be grouped into "User enters login and password."

- Use cases are not algorithms written in pseudo-code. They typically have a higher level of abstraction than algorithms. Use cases should be comprehensible to end-users, who should be able to read, understand, and identify issues in them. Thus, avoid statements like `if`, `repeat until`, etc. For example, instead of a repetition command, use a sentence like: "The customer browses the catalog until finding the desired product."

- Use cases should not address technology choices or design aspects. Moreover, they should not depend on the user interface that the main actor will use to interact with the system. For example, we should not write steps like: “The customer presses the green button to confirm the transfer.” It’s important to remember that we are specifying requirements, and decisions about technology, design, architecture, and user interface are not yet relevant. The objective is to document “what” the system should do, not “how” it will implement the steps.

- Avoid trivial use cases, such as those with only CRUD (Create, Retrieve, Update, and Delete) operations. For instance, in an academic system, it is not advisable to have use cases like Create Student, Retrieve Student, Update Student, and Delete Student. Instead, consider creating a use case like “Manage Student” and briefly explain that it includes these four operations. As the semantics are clear, this can be described in one or two sentences. Moreover, the main flow does not always need to be a list of actions. In some situations, such as the ones we are mentioning, it is more practical to use free text.

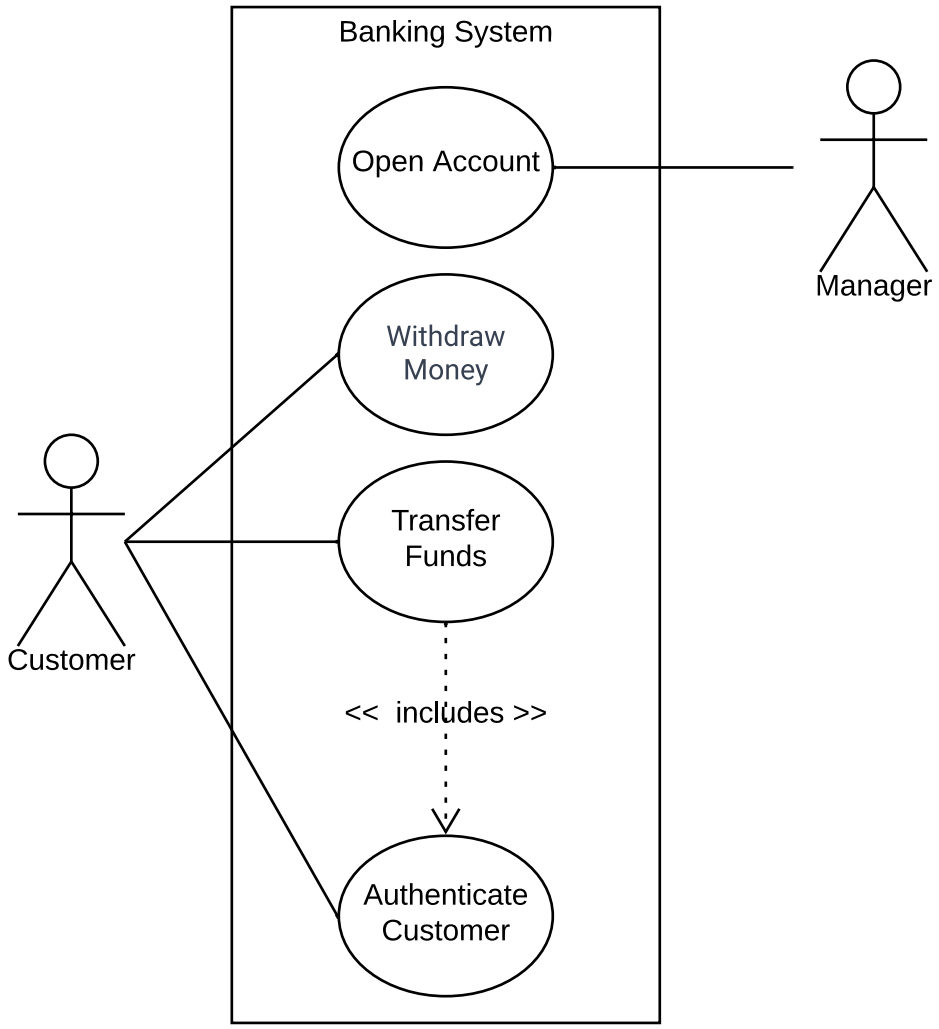
- Use a consistent vocabulary across use cases. For example, avoid using the name “Customer” in one use case and “User” in another. In the book *The Pragmatic Programmer* (link, page 251), David Thomas and Andrew Hunt recommend creating a **glossary**, i.e., a document that lists the terms and vocabulary used in a project. The authors state that “it’s hard to succeed on a project if users and developers call the same thing by different names or, even worse, refer to different things by the same name.”

3.4.1 Use Case Diagrams 🔗

In Chapter 4, we will study the UML graphical modeling language. However, we would like to anticipate and comment on one of the UML diagrams, known as the **Use Case Diagram**. This diagram serves as a “visual catalog” of use cases, depicting the actors of a system (illustrated as stick figures) and the use cases (depicted as ellipses). Additionally, it shows two types of relationships: (1) a line linking an actor to a use case indicates the actor’s participation in a given scenario; (2) an arrow linking two use cases indicates that one use case either includes or extends the other.

A simple use case diagram for our banking system is shown in the following figure. It features two actors: Customer and Manager. The Customer is involved in two use cases (“Withdraw Money” and “Transfer Funds”), while the Manager is the principal actor in the “Open Account” use case. The diagram also indicates that the “Transfer Funds” use case

includes “Authenticate Customer.” Lastly, we can observe that the use cases are depicted within a rectangle, which represents the system boundary. The two actors are situated outside this boundary.



Example of a UML Use Case Diagram

In-Depth: In this book, we distinguish between use cases (textual documents for specifying requirements) and use case diagrams (visual catalogs of use cases, as proposed in UML). Craig Larman makes this same distinction in his book about UML and design patterns (link, page 48). Larman asserts that “use cases are text documents, not diagrams, and use case modeling is primarily an act of writing, not drawing.” Martin Fowler expresses a similar view, recommending that we “concentrate our energy on the text rather than on the diagram. Despite the fact that the UML has nothing to say about the use case text, it is the text that contains all the value in the technique” (link, page 104).

3.4.2 Frequently Asked Questions 🔗

Let's now answer two questions about use cases.

What is the difference between use cases and user stories? A simple answer is that use cases are more detailed and comprehensive documents than user stories. A more elaborate explanation is provided by Mike Cohn in his book about user stories (link, page 140). According to Cohn, "use cases are written in a format acceptable to both customers and developers so that each may read and agree to them. Their purpose is to document an agreement between the customer and the development team. Stories, on the other hand, are written to facilitate release and iteration planning, and to serve as placeholders for conversations about the users' detailed needs."

What is the origin of the use case technique? Use cases were proposed in the late 1980s by Ivar Jacobson, one of the pioneers of UML and of the Unified Process (UP) (link). Use cases are one of the primary outputs of UP's Elaboration phase. As mentioned in Chapter 2, UP emphasizes written communication between customers and developers, using documents such as use cases.

3.5 Minimum Viable Product (MVP) 🔗

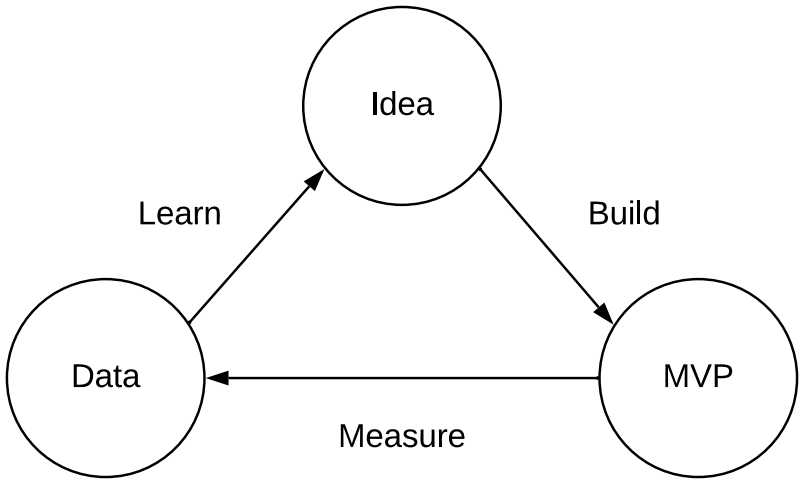
The concept of MVP was popularized by Eric Ries in his book *The Lean Startup* (link). This idea of Lean Startup was in turn inspired by the principles of the Lean Manufacturing movement, developed by Japanese automobile manufacturers, such as Toyota, since the 1950s. Kanban, as we studied in Chapter 2, is another software engineering technique based on this movement. One of the principles of Lean Manufacturing recommends eliminating waste in an assembly line or supply chain. For software companies, potential waste includes devoting years to gathering requirements and implementing a system that will not be used, because it solves a problem that is no longer relevant to users. Therefore, if a system is going to fail—by not being able to attract users or find a market—it's better to fail quickly, as the waste of resources will be less.

Software systems that do not attract interest can be produced by any company. However, they are more common in startups because, by definition, startups operate in environments of high uncertainty. On the other hand, the definition of a startup is not restricted to a company formed by two university students developing a new product in a garage. According to Ries (page 27 of his book), "anyone who is creating a new product or business under conditions of extreme uncertainty is an entrepreneur whether he or she knows it or not and whether working in a government agency, a venture-backed company, a nonprofit, or a decidedly for-profit company with financial investors."

To clarify our scenario, let's suppose we intend to create a new system, but we are not sure whether it will attract users or succeed. As noted earlier, in such cases, it is not recommended to spend years defining the requirements and implementing this system, only to then conclude it will be a failure. However, it also doesn't make sense to conduct market research to infer the system's reception. Because our requirements are different from those of any existing system, the results of this research may not be reliable.

Therefore, a solution is to implement a system with the minimum set of requirements that are sufficient to test the viability of its development. In the Lean Startup terminology, this initial system is referred to as a **Minimum Viable Product (MVP)**. An MVP's goal is commonly described as testing a business hypothesis.

Moreover, the Lean Startup movement proposes a systematic and scientific method for building and verifying MVPs. This method consists of a cycle with three steps: build, measure, and learn (see the next figure). In the first step (build), one has a product idea and implements an MVP to test it. In the second step (measure), the MVP is made available to real customers to collect data on its usage. In the third step (learn), the collected data is analyzed, resulting in what is termed validated learning.



MVP Validation

The knowledge derived from an MVP test can lead to the following decisions:

- We may conclude that further tests with the MVP are needed, possibly changing its requirements, user interface, or target market. In this case, the cycle is repeated, returning to the build step.
- We may conclude that the test was successful, and therefore a market for the system (a product-market fit) was found. Consequently, it's time to invest more resources to implement a feature-complete and robust system.

- Finally, the MVP might have failed after several attempts. This leaves two alternatives: (1) abandon the venture, particularly if there are no more financial resources to keep it alive; or (2) perform a **pivot**, which means abandoning the original vision and attempting a new MVP with major changes, such as a completely different set of features or targeting a new market.

One key risk when making these decisions is relying on **vanity metrics**. These are superficial metrics that serve to inflate the egos of developers and product managers while offering limited insight into enhancing the market strategy. A typical example is the number of page views on an e-commerce site. While attracting millions of monthly visitors may be satisfying, it won't necessarily translate to sales or profit.

In contrast, **actionable metrics** are those that can inform decisions about the MVP's future. In our e-commerce example, they include the conversion rate of visitors to buyers, the average order value, the number of items sold per transaction, and customer acquisition costs, among others. By monitoring these metrics, we might discover that customers typically purchase only one item per transaction. This finding could then prompt an actionable step, such as adopting a recommendation system. These systems suggest additional items during a transaction, potentially increasing the sales per order.

When assessing MVPs that involve product or service sales, **funnel metrics** are often used. These metrics measure the different levels at which users interact with a system. A typical funnel might be broken down as follows:

- Acquisition: Number of users who visited our system.
- Activation: Number of users who created an account.
- Retention: Number of users who returned after creating an account.
- Revenue: Number of users who made a purchase.
- Referral: Number of users who recommended the system to others.

3.5.1 MVP Examples 🔗

An MVP doesn't need to be software, implemented in a programming language, with databases, user interfaces, and integrations with external systems. Two frequently cited examples of MVPs that are not software systems appear in Lean Startup literature.

The first case is Zappos, one of the pioneering companies that attempted to sell shoes online in the United States. In 1999, to check the viability of an online shoe store, the company's founder conceived a simple and original MVP. He visited local stores,

photographed several pairs of shoes, and created a simple web page where customers could select the shoes they wanted to buy. All backend processing, however, was done manually, including payment processing, purchasing the shoes from local stores, and delivering them to customers. No system existed to automate these tasks. Despite this manual process, the company's founder quickly validated his initial hypothesis—namely, that there was indeed a demand for online shoe retail. Years later, Amazon acquired Zappos for over a billion dollars.

Dropbox, the cloud storage and file sharing service, provides another example of an MVP that did not involve making actual software available to users. To gather product feedback, one of the company's founders recorded a simple 3-minute video demonstrating the features and advantages of the system they were building. The video went viral, helping increase the list of users interested in beta-testing the system. Interestingly, the files used in this video had the names of comic book characters. This choice aimed to attract early adopters, enthusiasts about new technologies and who are typically the first to test and buy new products. The MVP's success confirmed the founders' hypothesis that users were interested in a file synchronization and sharing system.

However, MVPs can also be implemented as actual, albeit minimal, software apps. For example, in early 2018, our research group at UFMG started implementing an index of Brazilian papers in Computer Science. Our first decision was to build an MVP, covering only papers published in about 15 software engineering conferences. This initial version, implemented in Python, had fewer than 200 lines of code. The charts displayed by the MVP were simply Google Spreadsheets embedded in HTML pages. We initially called the index CoreBR and announced it on a mailing list for Brazilian software engineering instructors.

The index attracted significant interest, which we measured using metrics such as session duration. Based on this response, we decided to invest more time in its development. First, we changed the name to CSIndexbr (link). We then gradually expanded coverage to include an additional 20 research areas (beyond software engineering) and nearly two hundred conferences. Furthermore, we broadened our scope to include papers published in more than 170 journals. As a result, the number of Brazilian professors with indexed articles increased from fewer than 100 to over 900. Finally, we upgraded the user interface from a set of Google spreadsheets to JavaScript-implemented charts.

3.5.2 Frequently Asked Questions 🔗

To conclude, let's answer some questions about MVPs.

Should only startups use MVPs? No. As we've discussed in this section, MVPs are a mechanism for dealing with uncertainty. Specifically, when we don't know if users will like and use a particular product. In the context of software engineering, this product is software. On the one hand, startups, by definition, operate in markets of extreme uncertainty. On the other hand, risk and uncertainty can also be significant factors in software development across various types of organizations: private or public; small, medium, or large; and from diverse sectors.

When is it not worthwhile to use MVPs? In a way, this question was addressed in the previous one. When the market for a software product is stable and well-known, there is no need to validate business hypotheses and, therefore, no need to build MVPs. MVPs are also less common in mission-critical domains. For example, the idea of building an MVP to monitor ICU patients is unthinkable.

What's the difference between MVPs and prototypes? Prototyping is a well-known technique in software engineering for validating requirements. The distinction between prototypes and MVPs stems from the three letters of the acronym—M, V, and P. First, prototypes are not necessarily minimal systems. For example, they may include the entire interface of a system, with hundreds of screens. Second, prototypes are not necessarily used to check a system's viability in terms of market fit. For instance, they may be built to demonstrate the system only to the executives of a contracting company. Finally, prototypes are not products intended for use by any customer.

Is an MVP a low-quality product? This question is trickier to answer. On the one hand, an MVP should have only the minimum quality needed to evaluate a business hypothesis. For instance, the code doesn't need to be easily maintainable or to use the most modern design and architectural patterns. In fact, any level of quality above what is necessary to start the build-measure-learn feedback loop is wasteful. On the other hand, the quality shouldn't be so low that it negatively impacts the user experience. For example, if an MVP is hosted on a server with major availability issues, it might lead to false negatives. In other words, the business hypothesis may be falsely invalidated. In this case, the invalidation would not be due to the hypothesis itself, but rather to users being unable to access the system.

3.5.3 Building the First MVP 🔗

The Lean Startup approach doesn't specify how to construct the first MVP of a system. In most cases, this isn't a problem, as the developers and business people have a clear idea of the features and requirements that should be present in the MVP. Thus, they can quickly implement the first MVP and initiate the build-measure-learn cycle. However, in some cases,

the definition of this first MVP might not be clear. For such situations, it's recommended to build a prototype before implementing the first MVP.

Design Sprint is a method proposed by Jake Knapp, John Zeratsky, and Braden Kowitz for testing and validating new products using prototypes (link). The main characteristics of a design sprint—not to be confused with a Scrum sprint—are as follows:

- Time-boxed: A design sprint lasts five days, beginning on Monday and ending on Friday. The aim is to quickly discover an initial solution to a problem.
- Small and multidisciplinary teams: A design sprint brings together a multidisciplinary team of seven people. This number was chosen to encourage discussions; therefore, the team can't be too small. However, it also aims to prevent endless debates; thus, the team can't be too large. The team should include representatives from all areas involved with the problem under investigation, including marketing, sales, logistics, technology, etc. Additionally, and of equal importance, a decision-maker should be part of the team, such as the company owner or a C-level executive.
- Clear objectives and rules: The first three days of the design sprint aim to converge, then diverge, and, finally, converge again. Specifically, on the first day, the team discusses and defines the problem to be solved. The goal is to ensure that, in the following days, the team will focus on solving the same problem (convergence). On the second day, potential solutions are proposed freely (divergence). On the third day, a winning solution is selected from the possible alternatives (convergence). The decision-maker has the final word in this choice, as a design sprint is not a purely democratic process. On the fourth day, a prototype is implemented, which can be as simple as a set of static HTML pages, without code or functionality. On the last day, the prototype is tested with five real-world customers, who interact with it in individual sessions.

Before concluding, it's important to mention that design sprints are not only used to create MVPs. The technique can be used to propose solutions to various problems. For example, a design sprint can be organized to redesign the interface of an existing system or to improve services in a hotel.

3.6 A/B Testing 🔗

A/B Testing (or *split testing*) is used to choose between two versions of a system based on user interest. The two versions are identical except that one implements requirements A and

the other implements requirements B—both being mutually exclusive. The goal is to determine which set of requirements should be supported in the final system. To make this decision, versions A and B are released to distinct user groups. We then assess which version generates greater user interest. Thus, A/B Testing serves as a data-driven approach for selecting requirements or features. The requirements from the winning version remain in the system, while the other version is discarded.

A/B Testing is commonly applied in various scenarios. For instance, it can be used when comparing an existing MVP with requirements A to a new MVP with requirements B after a build-measure-learn cycle. It's also commonly employed for testing user interface components. For example, given two layouts of a website, an A/B test can be used to determine which one produces the best user engagement. We can also test various elements, such as the color or position of a button on the page, the wording of messages used, or the order of items in a list, among others.

To perform an A/B test, we need two versions of a system, which we will call the **control version** (original system, requirements A) and the **treatment version** (requirements B). As an illustration, consider an e-commerce system. The control version uses a traditional recommendation algorithm, while the treatment version uses a novel and optimized algorithm. In this scenario, an A/B test would help determine whether the new recommendation algorithm outperforms the original and should therefore be implemented in the system.

To conduct the A/B test, we also need a metric to measure the gains achieved with the treatment version. In our e-commerce example, this metric could be the percentage of purchases originating from recommended links. The expectation is that the new recommendation algorithm will increase this percentage.

To implement the A/B test, we need to configure the system so that half of the users experience the control version and the other half the treatment version. Importantly, these versions must be randomly assigned to users. For each user session, we randomly determine which version they will encounter, as in the following code:

```
version = Math.random(); // random number between 0 and 1
if (version < 0.5)
   "execute the control version"
else
   "execute the treatment version"
```

After a sufficient number of accesses, we should conclude the test and assess whether the treatment version has indeed increased the conversion rate. If the results are positive, we should implement the treatment version for all users. Otherwise, we should retain the control version.

The number of customers tested with each version, or sample size, is a vital aspect of A/B Testing. While the detailed statistical procedures for computing the size of this sample are outside the scope of this book, there are various A/B test sample size calculators available online. It's important to note that these tests typically require a large sample size, usually only achievable by popular platforms such as e-commerce sites, search engines, social networks, or news portals.

To illustrate, consider a scenario where the customer conversion rate is 1% for the control version, and we aim to test if the treatment version provides a minimum gain of 10% to this rate. In this scenario, to have statistically relevant results with a 95% confidence level, the control and treatment groups must have at least 200,000 customers each. To explain further:

- If after 200K accesses, version B increases the conversion rate by at least 10%, we have statistical confidence that this gain was caused by treatment B (in fact, we can be 95% confident). In this case, we conclude that the test was successful, and version B is the winner.
- Otherwise, if version B does not achieve the intended conversion rate, we conclude that the test has failed and version A remains the preferred option.

The sample size required by an A/B test considerably decreases when we test for higher conversion rates. For instance, if we modify our previous example such that the baseline conversion rate is 10% and we aim for a 25% improvement, the required sample size drops significantly to 1,800 customers for each group. These values were estimated using the A/B test calculator from Optimizely (link).

In-Depth: In statistical terms, an A/B Test is modeled as a **Hypothesis Test**. In such tests, we start with a Null Hypothesis that represents the "status quo." That is, the Null Hypothesis assumes that there is no significant difference between the current version (A) and the new version (B) of the system. The hypothesis that challenges the "status quo" is called the Alternative Hypothesis. Conventionally, we represent the Null Hypothesis as H_0 and the Alternative Hypothesis as H_1.

A Hypothesis Test is a decision-making procedure that starts with the assumption that H_0 is true and then attempts to find evidence against it. However, the statistical test used for this purpose is subject to a margin of error. For instance, it might reject H_0 even when it is correct. In such cases, we say a Type I error or false positive has occurred because we incorrectly concluded that there is a difference between versions A and B.

Though Type I errors cannot be entirely avoided, their probability can be estimated. In A/B tests, this probability is called the significance level, represented by the Greek letter α (alpha). It defines the probability of making a Type I error.

For example, if we set α at 5%, it implies a 5% chance of rejecting H_0 when it is actually true. In this book, rather than α, we use the parameter $(1 - \alpha)$, known as the confidence level, which represents the probability of correctly accepting H_0 when it is true. We focus on the confidence level as it is the most common input parameter in A/B test sample size calculators available online.

3.6.1 Frequently Asked Questions 🔗

Here are some questions and clarifications on A/B testing.

Can I test more than two variations? Yes, the approach we described adapts to more than two versions. Just divide the traffic into three random groups, for example, if you want to test three versions of a system. These tests, with more than one treatment, are called A/B/n tests.

Can I conclude the A/B test early if it shows the expected gain? No, this is a common and serious mistake. If the predetermined sample size is 200,000 users, the test—for each group—can only be concluded when we reach this number of users, to ensure statistical significance. A common mistake developers make when beginning to use A/B testing is to conclude the test on the first day the expected gain is reached, without testing the rest of the sample.

What is an A/A test? It's a test in which both the control and treatment groups execute the same version of the system. Therefore, assuming a 95% statistical confidence, they should almost always fail, as version A cannot be better than itself. A/A tests are recommended for validating the framework and procedures used in an A/B test. Some authors even recommend not starting A/B tests before performing A/A tests (link). If the A/A tests do not fail, we should debug the test setup to identify the root cause leading to the incorrect conclusion that version A is better than itself.

What is the origin of the terms control and treatment groups? The terms originate from the medical field, more specifically in randomized controlled experiments. For example, to introduce a new drug to the market, pharmaceutical companies must conduct this type of experiment. They choose two groups, called control and treatment. The participants in the control group receive a placebo, while the participants in the treatment group receive the drug. After the test, results are compared to assess the drug's efficacy. These experiments are a scientifically accepted method for establishing causality.

Real World: A/B tests are widely used by all major internet companies. Below, we present testimonials from developers and scientists at three companies about these tests:

- At Facebook (now Meta), "A/B testing is an experimental approach to finding what users want, rather than trying to elicit requirements in advance and writing specifications. Moreover, it allows for situations where users use new features in unexpected ways. Among other things, this enables engineers to learn about the diversity of users, and appreciate their different approaches and views of Facebook." (link)
- At Netflix, "if not enough people hover over a new element, a new experiment might move the element to a new location on the screen. If all experiments show a lack of interest, the new feature is deleted." (link)
- At Microsoft, specifically for the Bing search service, "the use of controlled experiments has grown exponentially over time, with over 200 concurrent experiments now running on any given day. The Bing Experimentation System is credited with having accelerated innovation and increased annual revenues by hundreds of millions of dollars, by allowing us to find and focus on key ideas evaluated through thousands of controlled experiments." (link)

Bibliography 🔗

Mike Cohn. User Stories Applied: For Agile Software Development. Addison-Wesley, 2004.

Alistair Cockburn. Writing Effective Use Cases. Addison-Wesley, 2000.

Eric Ries. The Lean Startup: How Today's Entrepreneurs Use Continuous Innovation to Create Radically Successful Businesses. Crown Business, 2011.

Jake Knapp, John Zeratsky, Braden Kowitz. Sprint: How to Solve Big Problems and Test New Ideas in Just Five Days. Simon & Schuster, 2016.

Ian Sommerville. Software Engineering. Pearson, 10th edition, 2019.

Hans van Vliet. Software Engineering: Principles and Practice. Wiley, 3rd edition, 2008.

Exercises 🔗

1. Mark True (T) or False (F).

( ) Requirements engineering, like other software engineering activities, needs to be tailored to the needs of the project, product, and teams.

( ) When gathering and analyzing requirements, developers collaborate with stakeholders to gain knowledge about the application domain, system requirements, performance standards, hardware constraints, and more.

( ) As the collected information comes from various perspectives, the emerging requirements are always consistent.

( ) Requirements validation involves confirming whether the requirements accurately define the intended system. This process is critical because errors in a requirements document can lead to significant rework costs.

2. List at least five methods for eliciting requirements.

3. What are the three parts of a user story? Describe your answer using the 3C acronym.

4. Consider a social network like Instagram: (1) Write five user stories for this network from the perspective of a typical user; (2) Now, think of another user role and write at least two stories related to it.

5. In software engineering, anti-patterns are non-recommended solutions for a certain problem. Describe at least five anti-patterns for user stories. In other words, describe story patterns that are not recommended or that do not have desirable properties.

6. Specify an epic user story for a system of your choice.

7. In the context of requirements, define the term Gold Plating.

8. Write a use case for a Library Management System (similar to the one we used in Section 3.3.1).

9. The following use case has only the main flow. Write some extensions for it.

> **Buy Book**
>
> **Actor**: Online store user
>
> **Main Flow:**
>
> 1. User browses the book catalogue
> 2. User selects books and adds them to the shopping cart
> 3. User decides to checkout
> 4. User informs delivery address
> 5. User informs type of delivery
> 6. User selects payment mode
> 7. User confirms order

10. For each of the following requirements specification and/or validation techniques, describe a system where its use is appropriate: (1) user stories; (2) use cases; (3) MVPs.

11. How does a Minimum Viable Product (MVP) differ from the first version of a product developed using an agile method, such as XP or Scrum?

12. The paper *Failures to be celebrated: an analysis of major pivots of software startups* (link) covers nearly 50 software startup pivots. In Section 2.3, the paper classifies common types of pivots. Read this section, identify at least five pivot types, and provide a brief explanation of each.

13. Suppose we're in 2008, before Spotify existed. You decided to create a startup to offer a music streaming service on the Internet. Thus, as a first step, you implemented an MVP.

a. What are the core features of this MVP?

b. What hardware and operating system should the MVP be developed for?

c. Draw a simple sketch of the MVP's user interface.

d. What metrics would you use to assess the success or failure of your MVP?

14. Assume you are managing an e-commerce system. In the current system (version A), the call-to-action button reads “Add to Cart.” You plan to conduct an A/B test with a new message, “Buy Now” (version B).

    a. What metric would you use for the conversion rate in this test?

    b. If the original system has a conversion rate of 5% and you want to test a 1% increase with the new message (version B), what should the sample size be for each version? To answer this, use an A/B test sample size calculator, like the one suggested in Section 3.6.

All rights reserved. Version for personal use only. To report errors or typos use this form.